

Kubernetes Microservices

Kubernetes services provide a way to expose a set of pods as a network service with a stable IP address or DNS name. This simplifies communication between components, as pods' IPs are dynamic and can change frequently. Services also enable load balancing across pods.

Service Types:

1. **ClusterIP:**
 - Default service type if none is specified.
 - Creates a service with a stable IP address that is **accessible only within the Kubernetes cluster**.
 - Example: A service named **learning-service** routes traffic to pods with the label **app: learning-resources**. Traffic is routed from port 80 on the service to port 3000 on the pods.
 2. **NodePort:**
 - Exposes a service on a **specific port (NodePort)** of each Kubernetes node's IP address.
 - Requests to the node's IP and NodePort are forwarded to the service, which load balances traffic to the pods.
 - Example: A service named **echo-service** routes traffic to pods with the label **app: echo-server**. If no port is specified, Kubernetes assigns a port between 30000-32767.
 - **Security Note:** NodePort services expose pods to the internet, so they need careful consideration of security.
 3. **LoadBalancer:**
 - Integrates with cloud providers (AWS, GCP, Azure) to provision an **external load balancer**.
 - Traffic to the load balancer's public IP is routed into the cluster and distributed to pods.
 - Example: A **frontend** service directs traffic to pods labeled **app: frontend-ui**.
 4. **ExternalName:**
 - Maps a service to an **external DNS name** or IP address, allowing traffic to be routed to resources outside the cluster.
 - Example: A service named **database-service** routes traffic to **my.postgres.database.com** via its **externalName** configuration.
-

Summary:

- **ClusterIP:** Internal-only communication within the cluster.
- **NodePort:** Exposes services to the internet via node IPs and a specific port.

- **LoadBalancer:** Creates a cloud-based load balancer to expose services externally.
- **ExternalName:** Redirects traffic to external DNS names or IPs, bypassing pod selectors.

Each service type is designed to meet different networking needs in a Kubernetes cluster, ranging from internal communication to exposing workloads to the public.

Set up cluster and deploy microservices

1) Create cluster with calico:

- `uname -p`
- `minikube start --network-plugin=cni --cni=calico -p demo`
- `kubectl get nodes`
- `kubectl get pods -A`

Calico is an open-source networking and network security solution for containers, virtual machines, and native host-based workloads. It is widely used in Kubernetes environments to provide networking, network policies, and security features. Calico operates at Layer 3 (network layer) and can handle routing between pods, enabling scalable and performant communication.

CNI Plugin: Integrates with Kubernetes as a Container Network Interface (CNI) plugin, enabling pod networking.

Network Policies: Offers fine-grained control over traffic between pods and external services.

Performance: Uses BGP (Border Gateway Protocol) for scalable routing.

The cluster with calico will support advanced features like: - Pod IP routing and connectivity. - Isolation of pods using Calico network policies. - Enhanced performance with scalable networking

2) Deploy Backend services:

- `kubectl apply -f learning-resources-api.yaml` - One deployment and one Cluster IP service.
- `kubectl apply -f echo-server.yaml` - One deployment and one NodePort service.
- `kubectl get pods` - See 5 pods - 3 Cluster IP services and 2 NodePort services.
- `kubectl get svc` - Result will be 3 services. One for each applu and the default kubernetes Cluster IP one.

3) Deploy Frontend services:

- `kubectl apply -f frontend-ui.yaml` - One namespace, One deployment and one LoadBalancer service.
- `kubectl -n frontend get pods`
- `kubectl -n frontend get svc`

Send http from busybox service to Cluster IP service learning-resources

- 4) Deploy busybox pod
 - `kubectl apply -f busybox.yaml`
- 5) Insert to busybox container
 - `kubectl exec -it busybox -- sh`
- 6) Make http request from IP, from service name, from DNS server: General Terminal:
 - `kubectl get pods -o wide`
 - `kubectl get svc`

BusyBox terminal: - `wget -O- <ip_address>` - `wget -O- learning-service`
 - `wget -O- http://learning-service.default.svc.cluster.local`

Expose a group of Pods to Internet directly via NodePort service

When you create a NodePort service, Kubernetes opens a port on one or more of the nodes and it allows in at a specific port number.

- 7) Make http request to the NodePort service from IP, from service name, from DNS server: General Terminal:
 - `kubectl get nodes -o wide` BusyBox terminal:
 - `wget -O- <node_ip>:30076`
 - `wget -O- echo-service`
 - `wget -O- http://echo-service.default.svc.cluster.local`

Examine a LoadBalancer Service

A LoadBalancer service in Kubernetes allows exposing a group of Pods to the internet using a cloud provider's load balancer.

1. **Deployment and Service Setup:**
 - Apply the frontend-ui.yaml file:
`kubectl apply -f frontend-ui.yaml`

This creates:

- A namespace (**frontend**).
- A deployment (**frontend-ui**) with three Pods.
- A LoadBalancer service (**frontend-service**) targeting Pods with the label **app=frontend-ui**.

2. Service Details:

- Verify the service details:
`kubectl get services -n frontend`
- The **frontend-service** is of type **LoadBalancer**, with a **ClusterIP** and an external IP in **Pending** status (as this example uses Minikube, which doesn't support automatic external IP allocation).

3. Query the Service:

- Create a BusyBox pod in the same namespace:
`kubectl apply -f busybox-ui.yaml`
- Exec into the BusyBox pod:
`kubectl -n frontend exec -it busybox-ui -- sh`
- Query the service using its name and DNS:
`wget -O- frontend-service`
`wget -O- frontend-service.frontend.svc.cluster.local`
- The output confirms the service is functioning, returning HTML data from the frontend Pods.

4. Simulate External Access with Port Forwarding:

- Retrieve Pod names in the **frontend** namespace:
`kubectl get pods -n frontend`
- Port forward one of the Pods to access it locally:
`kubectl port-forward <pod_name> 8080:4173 -n frontend`
- Open the app in a browser at <http://localhost:8080>, which combines frontend and backend data into a webpage.

5. Using Minikube for LoadBalancer Access:

- Minikube supports LoadBalancer services via specific configurations. Refer to the Minikube documentation under “LoadBalancer Access” for details.

6. Kubernetes Learning Resources:

- Suggested resources include Kubernetes networking courses on LinkedIn Learning, KubeCon talks, and TechWorld with Nana.

This exercise demonstrates the creation and use of a LoadBalancer service and how to access services locally when an external IP is unavailable.